

Lab 1: Setup for: CIT 3107: Back-End Development

Welcome to the practical guide for setting up a modern FastAPI backend environment! This guide is tailored to get your computers configured cleanly, whether they are running **Windows** or **Linux (e.g., Ubuntu)**, without tying them to any specific text editor.

Step 0: Prerequisites Check

Before setting up the project environment, verify that you have Python and Git installed. use terminal (**PowerShell** on Windows or **Terminal** on Linux) and run:

Requirement	Windows Command	Linux Command	Expected Output
Python 3.12+	python --version	python3 --version	Python 3.12.x or 3.13.x
Git	git --version	git --version	git version 2.x.x

 **Note for Students:** If these commands return an error saying "command not found", you must download and install them from python.org and git-scm.com before moving forward.

Step 1: Install the UV Package Manager

Instead of using traditional pip and venv, we will use **UV**, a modern package manager written in Rust. It is up to 100x faster and simplifies project management.

Windows (PowerShell)

Run the following command as a normal user (or as Administrator if policies are strict):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Linux (Ubuntu/Debian)

Run the following curl command in your terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

 **Crucial Step:** Once the installation finishes, students **must close their terminal and open a new one** to refresh their system paths. Verify the installation by running `uv --version`.

Step 2: Create a Project Directory & Virtual Environment

Now, create an isolated workspace for our backend course so it doesn't conflict with any other Python projects on the computer.

1. Create and Enter the Folder

- **Windows (PowerShell):**

```
mkdir ~\cit-backend-course  
cd ~\cit-backend-course
```

- **Linux (Terminal):**

```
mkdir ~/cit-backend-course  
cd ~/cit-backend-course
```

2. Initialize the Virtual Environment

Inside your newly created directory, generate an isolated Python environment using UV:

```
uv venv
```

3. Activate the Environment

Activating tells your command line to look inside this folder for Python packages rather than globally.

- **Windows (PowerShell):**

```
.venv\Scripts\Activate.ps1
```

- **or Windows (Classic Command Prompt / CMD):**

```
.venv\Scripts\activate.bat
```

- **Linux:**

```
source .venv/bin/activate
```

 **How to know it worked:** Students should look at their command line prompt. It will now be prefixed with (.venv), meaning they are safely inside their isolated environment.

Step 3: Install FastAPI and Uvicorn

With the environment activated, install the core framework (**FastAPI**) and the lightning-fast web server (**Uvicorn**).

Run this single command:

```
uv add fastapi uvicorn
```

What just happened? uv add downloads the packages instantly and populates a tracking file called pyproject.toml. This ensures your project dependencies are clearly defined.

Step 4: Write Your First API (main.py)

use any text editor you prefer (e.g., **Notepad** on Windows, **Gedit** or **Nano** on Ubuntu) to create this file.

open the editor, create a new file named **main.py** inside the cit-backend-course folder, and write the following 4 lines of code:

```
from fastapi import FastAPI

app = FastAPI(title="My Backend API")

@app.get("/")
def root():
    return {"message": "Hello from CIT Backend Course!"}
```

Quick Code Breakdown:

- `from fastapi import FastAPI`: Imports the web framework.
- `app = FastAPI(...)`: Initializes your core web application engine.
- `@app.get("/")`: A decorator stating: "When a browser requests the root URL (/) via HTTP GET, trigger the function right below."
- `return {...}`: Returns a native Python dictionary, which FastAPI automatically renders into clean, industry-standard **JSON** format.

Step 5: Launch the Server

Now, let's fire up our web application using the Uvicorn application server. Run this command inside your terminal:

```
uv run uvicorn main:app --reload
```

Why these flags?

- `uv run`: Forces the system to run the command explicitly utilizing our local virtual environment.
- `main:app`: Instructs Uvicorn to look inside the **main.py** file for the variable object named **app**.
- `--reload`: Puts the server into development mode. Every time a student saves a change in their code, the server automatically reboots itself instantly.

Step 6: Verify and Explore Swagger UI

Once the terminal outputs that the application is running, open any web browser and navigate to the interactive API documentation automatically provisioned by FastAPI:

👉 <http://127.0.0.1:8000/docs>

You meet a clean, interactive user interface where you can click "**Try it out**" on the root endpoint, execute the request, and read the live JSON response straight from their machine!

Exercise:

1. you have created an environment called `venv`. Create a second one called your surname, and repeat the rest of the process. Delete `venv`.

2. The Port Switch (Infrastructure Basics)

By default, Uvicorn launches on port 8000. However, in the real world, ports are frequently blocked or occupied by other applications.

stop the current server (Ctrl + C) and relaunch it so it runs on port 9000 instead.

```
uv run uvicorn main:app --reload --port 9000
```

* **Verification:** prove it works by navigating to

`[http://127.0.0.1:9000/docs](http://127.0.0.1:9000/docs)` in their browser.